Reducing Hallucinations in Large Language Models Through a Neuro-Symbolic Reasoning Layer

A neuro-symbolic reasoning and verification layer for enterprise decision support

PROJECT REPORT

Pravin Anbalagan

Athiban Karthik

Vellore Institute of Technology (VIT), Vellore, India

System: PRAXIS · 28 October 2026 · source commit 8138fd2

Source code: https://github.com/A-Pravin02/praxis-neurosymbolic

1. Abstract

Retrieval grounds a language model in documents but does not stop it asserting things the documents do not support. This project builds PRAXIS, an architecture in which a language model is never the authority for a decision: a neural encoder interprets the question, a three-valued symbolic reasoner applies versioned business rules to authorized records, and a suite of 11 deterministic checks gates the answer before release. In the default configuration the decision path contains no language model at all.

On 255 held-out questions over five generated enterprise worlds, PRAXIS answers **0.796** correctly at a hallucination rate of **0.149** with **0.943** coverage, against 0.200 and 0.302 for the same corpus retrieved and answered by a 7B instruction-tuned model. The reduction is not bought with abstention: PRAXIS covers 0.943 of answerable questions where retrieval covers 0.462.

A web interface was built to make the pipeline inspectable stage by stage, and ten sample cases - one per question category - were run through it. **7 of 10 were answered correctly.** The three failures are reported here rather than omitted, because one of them is the architecture's central limitation: verification establishes that a claim follows from the evidence, not that it answers the question asked.

2. Introduction

2.1 Problem statement

Language models now answer questions that carry consequences: whether a refund is owed, which policy version governs a dispute, how much a traveller may reclaim. Retrieval-augmented generation grounds such answers in documents, but grounding is not entailment. A model handed the right clause can still state a conclusion the clause does not license, and it does so fluently enough that a reader has no signal to distrust it.

The common remedies keep the model as the authority. Self-consistency sampling, self-critique and chain-of-verification all ask the model to audit its own output - auditor and author are the same model, so a confident systematic error is invisible to both. Our own measurement bears this out: asking a 7B model to check its own retrieval-grounded draft moved accuracy from 0.200 to 0.247 and hallucination from 0.302 to 0.282 - an improvement far smaller than the error it is meant to remove.

2.2 Objectives

- Design an architecture in which the decision is established symbolically, so that the class of errors in which a model asserts an unsupported conclusion cannot arise on the decision path.
- Implement three-valued reasoning so that a fact the company has no record of is treated as unknown rather than false.
- Implement a deterministic evidence gate that withholds any answer whose claims are not supported by the retrieved records.
- Evaluate against retrieval and language-model baselines on the same questions, corpus and authorization context.
- Build an interface that exposes each stage of the pipeline, so the reasoning can be inspected rather than taken on trust.
- Report coverage beside every accuracy, so that abstention is never mistaken for reliability.

2.3 Scope

This is an architecture project evaluated on generated enterprise worlds with gold answers from an independent oracle. It is not a general-domain factuality result. The worlds are synthetic and are not real

customer traffic. No claim is made that the approach eliminates hallucination, or that it transfers to domains whose governing rules cannot be written down.

3. Literature Survey

Area	Representative work	Limitation this project addresses
Hallucination surveys	Ji et al. (2023); Huang et al. (2025)	Separate faithfulness from factuality; our target is narrower - a decision stated without entailment from the governing rule.
Retrieval-augmented generation	Lewis et al. (2020)	Supplies evidence but leaves inference to the model, so a grounded answer can still be unlicensed.
Self-critique and resampling	Chain-of-verification (2024); SelfCheckGPT (2023); Self-RAG (2024)	Auditor and author are the same model; a confident systematic error is invisible to both.
Claim-level scoring	FActScore (2023)	An evaluation method, not a release gate - it scores text after the fact rather than withholding it.
Neuro-symbolic solvers	Logic-LM (2023); LINC (2023); PAL (2023); Binder (2023)	Formalise a reasoning chain for a puzzle. PRAXIS formalises the enterprise's own rule base, versioned over time and filtered by an authorization view.

Table 1. Related work and the gap addressed.

4. System Architecture

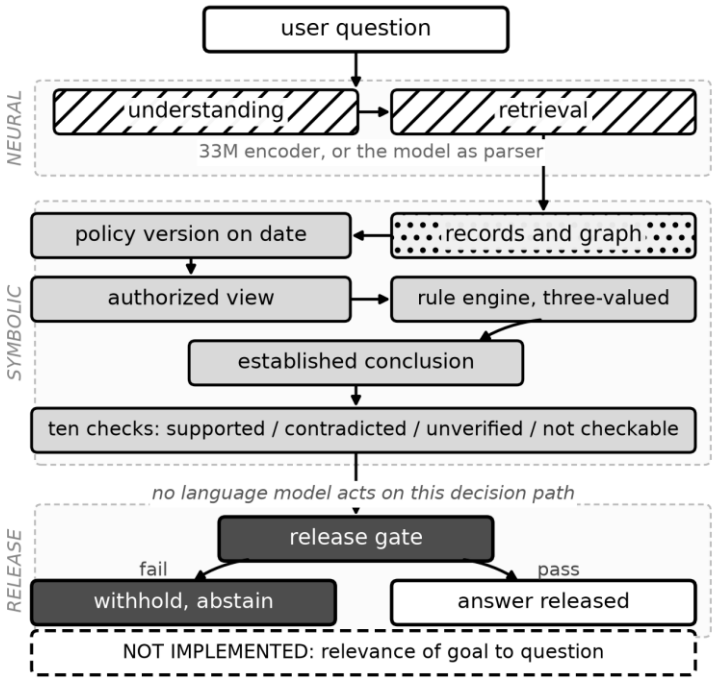


Figure 1. PRAXIS as implemented. Hatched blocks are neural, dotted stored knowledge, grey symbolic, dark the release gate. The dashed panel marks the one check that is absent.

The pipeline has four stages. The neural lane interprets the question and retrieves evidence; the symbolic lane resolves the policy version in force on the case's own date, restricts records to those the asker may see, and evaluates the rules under three-valued logic; the verifier checks the claims in the drafted reply against the evidence; the release gate publishes or withholds.

4.1 Neural understanding

A 33M-parameter bge-small encoder, run locally through ONNX, maps the question to a goal in the rule base, the records it refers to and any facts it states. The output is validated against the schema's own

declarations, so an interpretation naming a goal that does not exist is rejected rather than carried forward. The encoder proposes; it never asserts the answer.

4.2 Three-valued symbolic reasoning

Rules are evaluated under Kleene strong three-valued logic: a fact the company has no record of is unknown, not false. A two-valued engine answers 'no, you are not eligible' when the truthful answer is 'nobody recorded whether you are'. The reasoner resolves the policy version in force on the case's own date rather than the newest, and reads records through an authorized view, so a rule consults only facts the asker may see.

4.3 The evidence gate

The verifier defines **11 deterministic checks**: answer agreement, authorization, calculation, contradictions, fact validity, policy applicability, policy version, record consistency, supported numbers, symbolic provenance, unsupported claims. Which of them apply depends on what the case asserts, so none runs on every question. Measured over the 255 test cases, the number that runs ranges from 0 to 9, with a median of 8; the 17 cases that run none are refusals settled before there is anything to verify. Verdicts are four-valued - supported, contradicted, unverified, not checkable - so missing evidence is never read as refutation.

5. Implementation

5.1 Technology stack

Layer	Technology	Role
Neural encoder	bge-small (33M), ONNX Runtime	Question interpretation, goal and entity resolution; runs locally on CPU.
Symbolic reasoner	Python, custom rule engine	Kleene three-valued evaluation over versioned rules.
Knowledge	Typed knowledge graph over generated worlds	Records, policy history and an authorization view.
Verification	Python, deterministic checks	The 11-check suite and the release gate.
Backend	FastAPI (production), stdlib HTTP (demo)	The demo server needs no database, so it runs on a clean checkout.
Interface	HTML, CSS, vanilla JavaScript	Single page; no build step and no framework dependency.
Baselines	Qwen2.5-7B-Instruct via transformers	LLM-only, LLM+RAG, self-check and rule-extraction baselines.
Study II model	nanoGPT, 4.3M parameters, trained from random init	A deliberately weak generator of checkable claims.

Table 2. Implementation stack.

5.2 Repository structure

```
backend/app/hybrid/          controller, verification, grounding, knowledge pack
backend/app/symbolic/        three-valued rule engine
backend/app/neural/          encoder interpretation and semantic parser
backend/app/evaluation/      benchmark, scoring, system definitions
backend/data/benchmark_worlds/   13 generated worlds, 663 cases
demo/server.py               demo API driving the real pipeline
demo/static/index.html       the interface
scripts/                     experiment, audit and figure scripts
eval_reports/research/       per-question records for every reported number
docs/paper/                  the IEEE paper and its figures
```

5.3 The demo interface

The interface exists to make the pipeline inspectable. Every panel is filled from the controller's own trace - the same object the benchmark scores - so nothing shown on the page is precomputed or illustrative. A case that fails in the interface fails in the benchmark.

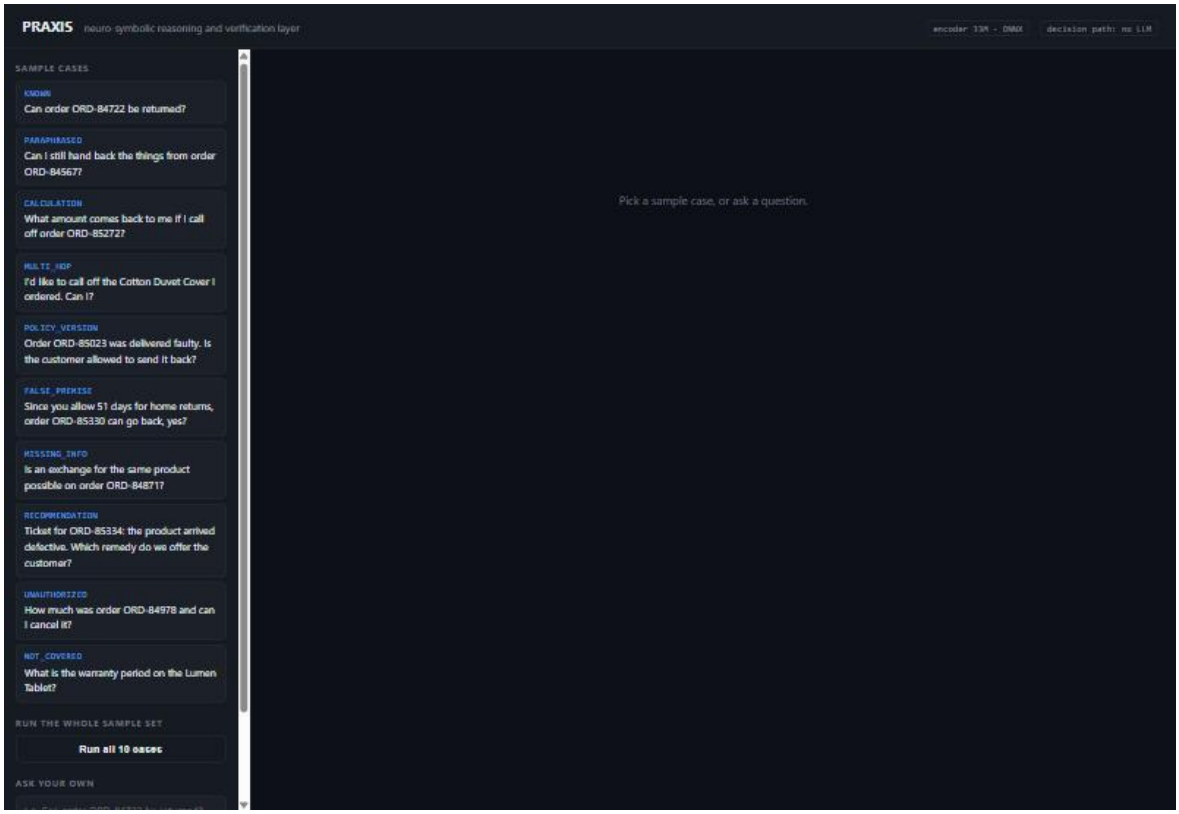


Figure 2. The interface on load. The left pane lists one sample case per question category; a question can also be typed directly.

6. Experimental Setup

Split	Worlds	Questions	Purpose
Development	3	153	Threshold calibration only; never reported.
Test (frozen)	5	255	Frozen before first use; main results.
Confirmatory	5	255	Written after the system was final; scored once.
Total corpus	13	663	All generated cases.

Table 3. Dataset splits. The three share no case identifier and no world.

Cases are generated over synthetic enterprise worlds, each with its own catalogue, staff, policy history and records, with gold answers from an independent oracle rather than the system under test. All systems see the same questions, corpus and authorization context; decoding is greedy, so a run repeats exactly on one machine.

Metrics. Let N be all cases, A the answerable cases - those whose gold kind is a decision, a value or a result - and D the subset of A answered definitely. Accuracy and hallucination are over N; coverage is |D|/|A|; selective accuracy is correctness over D. Coverage and selective accuracy are always reported together, because a system can buy high selective accuracy by refusing the hard questions.

7. Sample Test Cases and Results

Ten cases were run through the interface, one per question category, taken from world 101 of the frozen test split. The selection is the first case of each category, so it is reproducible and not chosen for effect. **7 of 10 were answered correctly** in 40 ms for the set.

Category	Question	Outcome	Gold kind	Result	ms
known	Can order ORD-84722 be returned?	answered	decision	**PASS**	5.5
paraphrased	Can I still hand back the things from order ORD-84567?	answered	decision	**FAIL**	2.3
calculation	What amount comes back to me if I call off order ORD-85272?	answered	value	**PASS**	2.4
multi_hop	I'd like to call off the Cotton Duvet Cover I ordered. Can I?	answered	decision	**PASS**	2.4
policy_version	Order ORD-85023 was delivered faulty. Is the customer allowed to s…	answered	decision	**PASS**	5.2
false_premise	Since you allow 51 days for home returns, order ORD-85330 can go b…	conditional	decision	**FAIL**	3.9
missing_info	Is an exchange for the same product possible on order ORD-84871?	conditional	conditional	**PASS**	5.3
recommendation	Ticket for ORD-85334: the product arrived defective. Which remedy …	answered	result	**PASS**	6.0
unauthorized	How much was order ORD-84978 and can I cancel it?	denied	denied	**PASS**	1.4
not_covered	What is the warranty period on the Lumen Tablet?	answered	not_covered	**FAIL**	5.2

Table 4. Sample test cases, run through the demo interface. 7/10 correct.

7.1 A case the system gets right

Case W101-T-known-1 asks whether an order can be returned. The encoder ranks candidate goals and carries forward the highest; the reasoner gathers the order's status, category and delivery date, applies the returns rules under three-valued logic, and establishes the conclusion before any text is written.

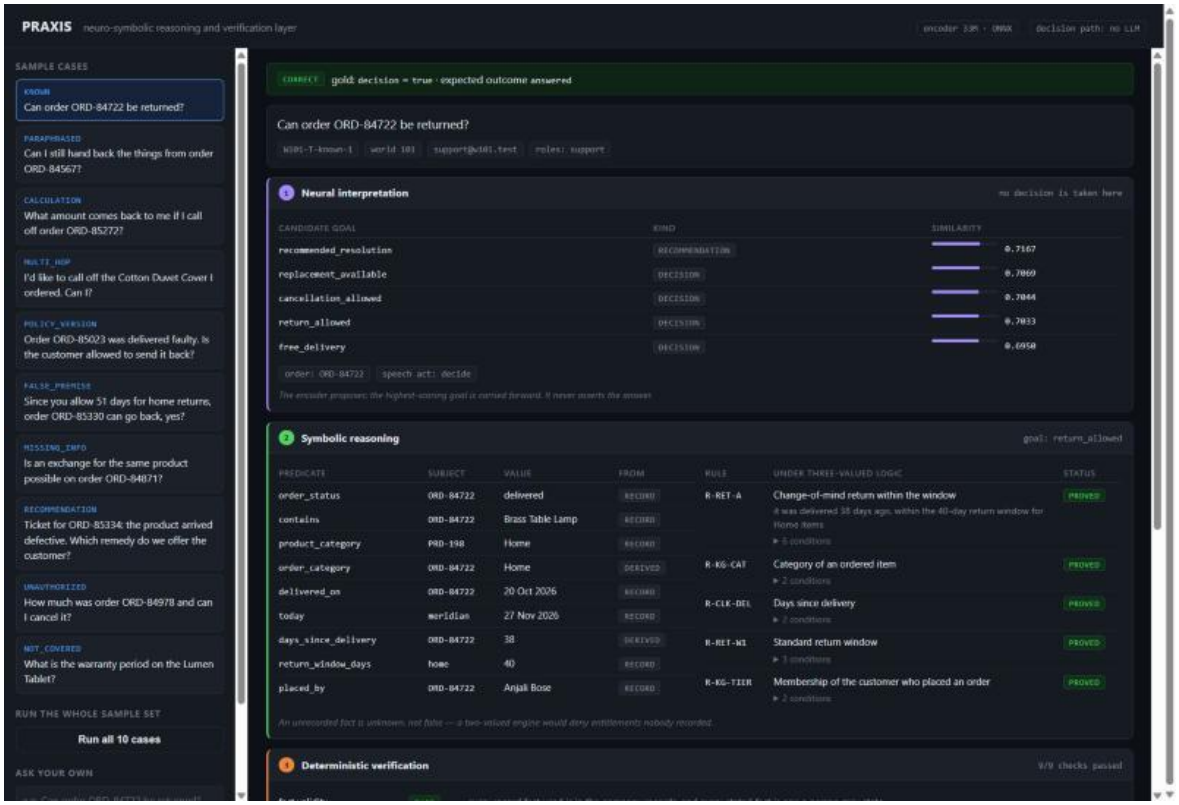


Figure 3. Stages 1 and 2. The encoder's candidate goals with similarity scores, and the symbolic facts and rules. R-RET-A is proved; R-RET-D is refuted - the damage window had passed - and the engine records both.

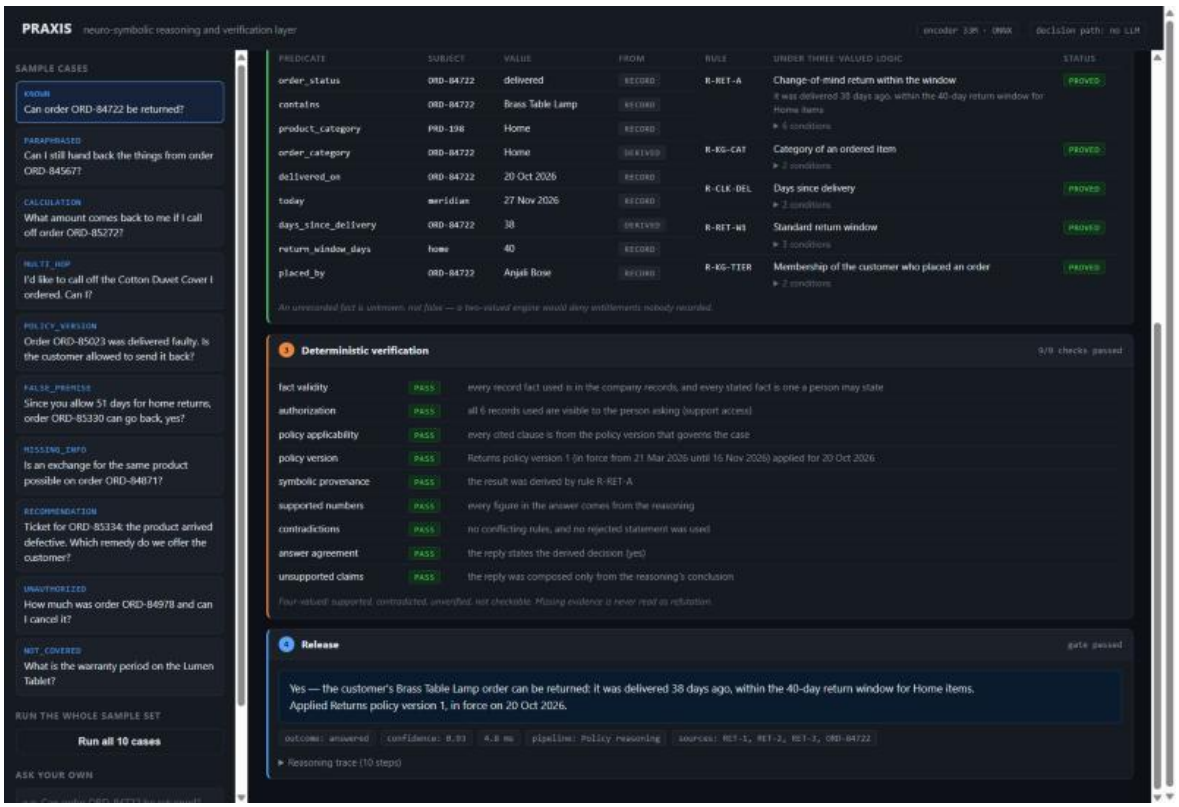


Figure 4. Stages 3 and 4. Nine checks applied to this case and all passed, each with the reason it passed. The released answer cites the policy version in force on the delivery date, not the newest.

7.2 The case the system gets wrong, and why it matters

Case W101-T-not_covered-1 asks for the warranty period on a product. No policy clause in the rule base covers warranties. The correct behaviour is to report insufficient evidence. Instead the encoder maps the question onto the nearest in-scope goal - replacement availability - and the system answers that question correctly, citing real clauses and real records.

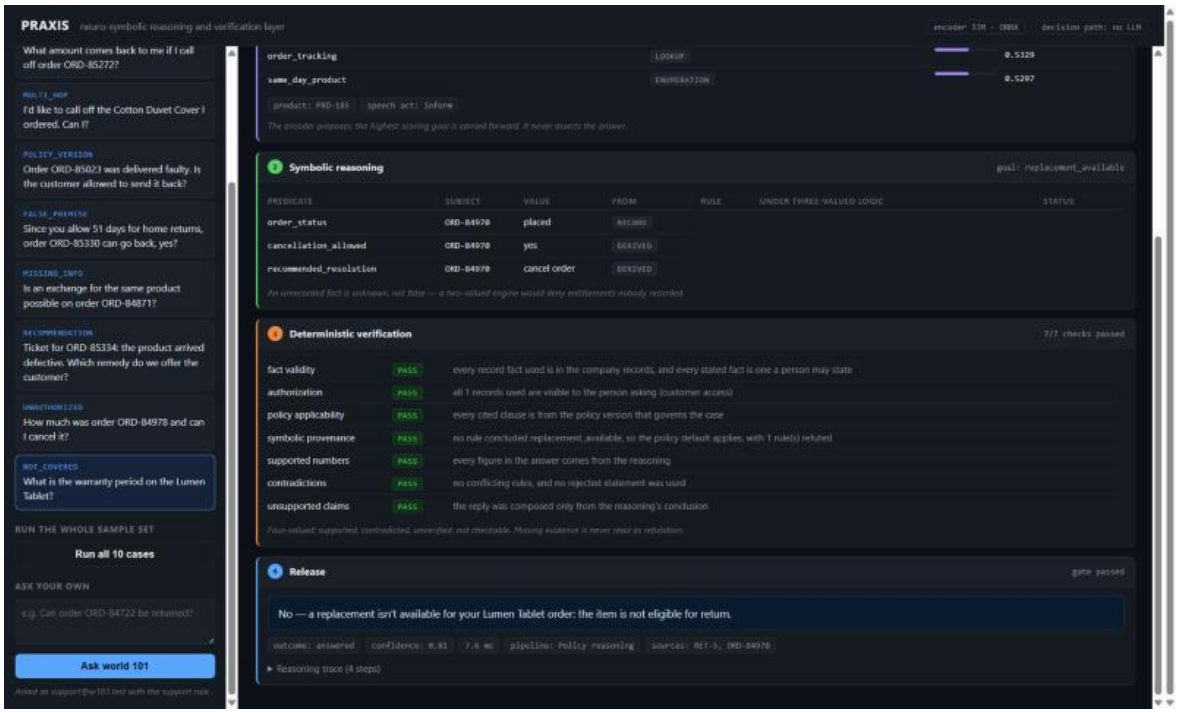


Figure 5. The documented failure. Asked about warranty, the system resolved replacement_available and answered confidently. Every applicable check passed, because every claim it made was supported.

The gate cannot intervene, and that is the finding. Verification establishes that a claim follows from the evidence, not that it answers the question asked. Across the fifteen not-covered questions in the test split the system abstained in none. A relevance check between the resolved goal and the question - the dashed panel in Figure 1 - is the remedy, and it is not implemented.

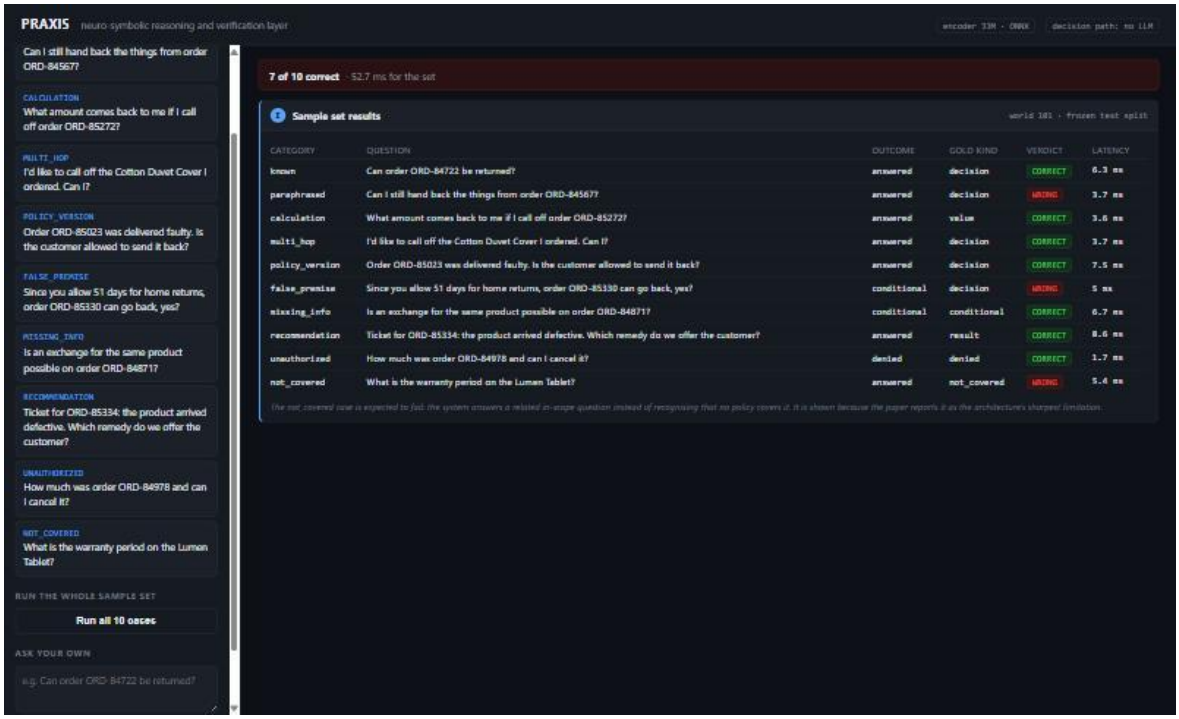


Figure 6. The full sample set run in one pass, with per-case verdicts and latencies.

8. Results and Discussion

8.1 Main comparison

System	Accuracy	Hallucination	Coverage	Selective acc.
PRAXIS (no LLM)	0.796	0.149	0.943	0.874
PRAXIS (LLM parser)	0.945	0.035	0.976	1.000
LLM + RAG + rules	0.537	0.247	0.781	0.793
LLM + RAG + self-check	0.247	0.282	0.505	0.481
LLM only	0.235	0.427	0.600	0.341
LLM + RAG	0.200	0.302	0.462	0.412
Extractive RAG	0.000	0.118	0.000	0.000

Table 5. Test split, 255 questions. Accuracy and hallucination are over all cases; coverage has a different denominator and is reported separately.

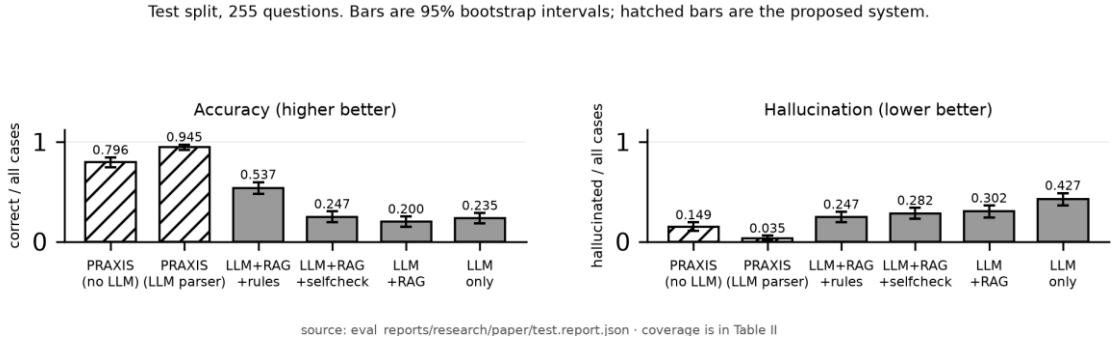


Figure 7. Accuracy and hallucination on the test split, with 95% bootstrap intervals.

The LLM-free configuration answers 0.796 of all questions correctly at 0.149 hallucination, against 0.200 and 0.302 for the same model given retrieval over the same corpus. The reduction is not bought with abstention: it covers 0.943 of answerable questions while retrieval covers 0.462 - it answers more questions and more of them correctly.

8.2 Ablations

Configuration	Accuracy	Hallucination	Coverage	Disclosure
PRAXIS (full)	0.796	0.149	0.943	0.000
Unknown treated as false	0.745	0.188	0.952	0.000
Newest policy applied to every case	0.761	0.184	0.943	0.000
Asker's statements override records	0.690	0.094	0.748	0.000
Reasoning sees every record	0.737	0.208	0.943	0.867

Table 6. Each row disables exactly one mechanism.

Three-valued reasoning carries real weight: treating unknown as false costs 0.051 accuracy, because the system starts denying entitlements that were merely unrecorded. Authorization is the sharpest: removing it leaves accuracy broadly intact but raises the disclosure rate from 0.000 to 0.867 - the system answers well and leaks almost every protected record while doing so, which an accuracy-only evaluation would not reveal.

One row needs reading with care. Letting the asker's statements override records lowers hallucination to 0.094, below the full system - but coverage falls from 0.943 to 0.748. The configuration abstains its way to a better hallucination rate, which is exactly why coverage is reported beside every accuracy.

8.3 Transfer to unseen worlds

On the confirmatory split - five worlds the system had never seen, written after it was final and scored once - the LLM-free configuration transfers essentially unchanged: 0.812 against 0.796 on test. The parser configuration does not: accuracy falls from 0.945 to 0.835, and selective accuracy from 1.000 to 0.909. The perfect selective accuracy on the test split does not generalize, and no claim of perfect reliability should be read from it.

8.4 Secondary study: gating a model trained from scratch

A 4.3M-parameter decoder was trained from random initialization on a corpus of FIFA World Cup goal records, then made to generate freely while the symbolic layer decided what reached the reader. Of 1,056 checkable claims, the graph refutes **493** (0.467) and cannot confirm a further 40; gating releases the 523 supported ones, leaving coverage of 0.495.

Refuted and unverifiable claims are kept apart deliberately: the knowledge graph is incomplete, so a claim it has no record of is not thereby false. The gated rate of zero refuted claims holds **by construction** - the gate releases only what the graph confirms - and is not evidence of perfect verification. The informative quantities are the base refuted rate and the coverage that suppressing it costs: verification buys soundness with silence.

9. Limitations

- **Scope.** Results come from generated worlds with oracle gold answers, not real customer traffic, in a single domain whose governing rules can be written down. English only.
- **Question relevance is not implemented.** On the fifteen not-covered questions the system abstained in none. This is the architecture's sharpest limitation and is demonstrated in Section 7.2.
- **Verifier correctness is unmeasured.** The verifier is correct by construction with respect to its own rule base; independent ground-truth labels would be needed to measure its precision and recall.
- **Authorship.** The rule base, world generators and oracle were written by one person. An independently authored evaluation is the natural next step.
- **Transfer.** Perfect selective accuracy on the test split does not generalize, as Section 8.3 shows.
- **Neither study speaks to open-domain factuality**, where no curated knowledge base exists for the gate to consult.

10. Conclusion and Future Work

Removing the language model from the decision path, rather than asking it to audit itself, reduced hallucination from 0.302 to 0.149 while raising coverage from 0.462 to 0.943 and cutting latency by roughly two orders of magnitude. The reduction is not an artefact of refusing difficult questions.

Two findings temper this. The parser configuration's perfect selective accuracy on the test split does not transfer to unseen worlds, and the system does not recognise questions its rule base does not cover - a failure verification is structurally unable to catch, because relevance is not support.

Future work is a relevance gate separate from the evidence gate, an independently authored evaluation, and raising coverage in the secondary study without reintroducing unsupported claims.

11. References

[1] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, pp. 1-38, 2023.

[2] L. Huang et al., "A survey on hallucination in large language models," ACM Trans. Inf. Syst., vol. 43, no. 2, 2025.

[3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in NeurIPS, vol. 33, 2020.

[4] A. Asai et al., "Self-RAG: learning to retrieve, generate, and critique through self-reflection," in ICLR, 2024.

[5] S. Dhuliawala et al., "Chain-of-verification reduces hallucination in large language models," in Findings of the ACL, pp. 3563-3578, 2024.

[6] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT," in EMNLP, pp. 9004-9017, 2023.

[7] S. Min et al., "FActScore," in EMNLP, pp. 12076-12100, 2023.

[8] L. Pan et al., "Logic-LM," in Findings of the ACL: EMNLP, pp. 3806-3824, 2023.

[9] T. X. Olausson et al., "LINC," in EMNLP, pp. 5153-5176, 2023.

[10] L. Gao et al., "PAL: program-aided language models," in ICML, PMLR vol. 202, pp. 10764-10799, 2023.

[11] Z. Cheng et al., "Binding language models in symbolic languages," in ICLR, 2023.

[12] A. d'Avila Garcez and L. C. Lamb, "Neurosymbolic AI: the 3rd wave," Artif. Intell. Rev., vol. 56, no. 11, pp. 12387-12406, 2023.

[13] S. C. Kleene, Introduction to Metamathematics. Amsterdam: North-Holland, 1952.

12. Appendix: Source Code and Reproduction

Repository: https://github.com/A-Pravin02/praxis-neurosymbolic

Commit reported here: 8138fd2

Running the demo interface:

```
python -m venv backend/.venv
backend/.venv/Scripts/pip install -r backend/requirements.txt
python demo/server.py --warm
# open http://127.0.0.1:8099
```

Reproducing the reported numbers (no database or GPU required):

```
python scripts/independent_metrics.py   # recomputes every metric
python scripts/verify_claims.py         # checks claims against artifacts
python scripts/count_checks.py          # censuses the verification checks
cd backend && python -m pytest -q -m no_db
```

The per-question records for every reported number are committed under `eval_reports/research/`, so a clone can re-verify the results without re-running the experiments. Model checkpoints are excluded for size and are regenerated by the training scripts.